

Machine Learning - Python +Pandas

July 10, 2026

1 PAO25_25_08_Python_+Pandas



Elvis Pachacama **Recopilado por:** Pablo Aguilar * Repositorio: GitHub

```
[1]: import pandas as pd
import numpy as np

# Línea 1: importa pandas con el alias estándar 'pd'.
# Línea 2: importa numpy con el alias estándar 'np', necesario para generar
↳ datos aleatorios y
#         usar funciones numéricas compatibles con pandas.
# Resultado esperado: no hay salida visible; ambas librerías quedan disponibles.
```

```
[2]: pd.__version__

# Línea 1: accede al atributo __version__ del módulo pandas; al ser la última
↳ expresión de la
#         celda, Jupyter la muestra automáticamente.
# Resultado esperado: un string con la versión instalada, por ejemplo '2.2.2'.
```

```
[2]: '3.0.3'
```

1.1 1. Operaciones en pandas

1.1.1 Búsqueda

```
[3]: rand_matrix = np.random.randint(6,size=(2,3))
frame = pd.DataFrame(rand_matrix , columns=list('ABC'))
display(frame)

# Línea 1: genera una matriz 2x3 de enteros aleatorios en el rango [0,6).
# Línea 2: construye un DataFrame a partir de esa matriz, nombrando las
↳ columnas 'A','B','C'
# (list('ABC') convierte el string 'ABC' en la lista ['A','B','C']).
# Línea 3: muestra la tabla resultante.
# Resultado esperado: una tabla 2x3 con enteros aleatorios entre 0 y 5 (valores
↳ distintos en
# cada ejecución) y columnas A, B, C.
```

	A	B	C
0	0	3	1
1	1	3	0

```
[4]: # buscando columnas (DataFrame como dic, busca en claves)
'A' in frame

# Línea 1: comentario descriptivo.
# Línea 2: el operador 'in' sobre un DataFrame comprueba si existe una COLUMNA
↳ con ese nombre
# (igual que comprobar si una clave existe en un diccionario, ya que
↳ las columnas
# funcionan como claves); al ser la última expresión de la celda, se
↳ muestra el resultado.
# Resultado esperado: True (la columna 'A' existe en 'frame').
```

[4]: True

```
[5]: # buscando valores
display(frame.isin([3,2])) # --> mask de respuesta (valores que son 3 o 2)

# Línea 1: comentario descriptivo.
# Línea 2: isin([3,2]) devuelve un DataFrame booleano del mismo tamaño que
↳ 'frame', con True en
# cada posición cuyo valor original es 3 o 2, y False en el resto.
# Resultado esperado: una tabla 2x3 de valores True/False según coincidan o no
↳ con 3 o 2.
```

	A	B	C
0	False	True	False
1	False	True	False

```
[6]: # Contar el número de ocurrencias
print(frame.isin([4]).values.sum())
display(frame.isin([4])) #devuelve una mask con valores true si el elemento es 4
print(frame.isin([4]).values)
type(frame.isin([4]).values) # pandas es una capa alrededor de numpy

# Línea 1: comentario descriptivo.
# Línea 2: isin([4]) genera la máscara booleana de dónde el valor es 4; .values
    ↳ la convierte en
#         un ndarray de NumPy, y .sum() suma los True (interpretados como 1)
    ↳ para contar cuántas
#         veces aparece el valor 4 en todo el DataFrame.
# Línea 3: comentario descriptivo.
# Línea 4: muestra la máscara booleana completa como tabla.
# Línea 5: imprime esa misma máscara ya convertida a ndarray de NumPy (formato
    ↳ de array plano,
#         no de tabla con índice/columnas).
# Línea 6: type() sobre .values confirma que el resultado subyacente es un
    ↳ numpy.ndarray,
#         evidenciando que pandas se construye internamente sobre NumPy; al
    ↳ ser la última
#         expresión de la celda, el tipo se muestra automáticamente.
# Resultado esperado: un entero con el conteo de apariciones de 4 (puede ser 0
    ↳ si no hay ningún
#         4 en la matriz aleatoria), la tabla booleana, el array booleano
    ↳ equivalente, y
#         finalmente numpy.ndarray como tipo.
```

0

	A	B	C
0	False	False	False
1	False	False	False

```
[[False False False]
 [False False False]]
```

[6]: numpy.ndarray

```
[7]: # Cuántos valores son >= 2
mask = frame >= 2
print(mask.values.sum())
display(mask)

# Línea 1: comentario descriptivo.
# Línea 2: comparar el DataFrame con un escalar (>=2) genera una tabla booleana
    ↳ del mismo
#         tamaño, con True donde se cumple la condición.
```

```
# Línea 3: convierte la máscara a ndarray y suma los True para contar cuántos
↳ valores cumplen
#           la condición (>=2) en todo el DataFrame.
# Línea 4: muestra la máscara booleana completa.
# Resultado esperado: un entero con el conteo de valores mayores o iguales a 2,
↳ seguido de la
#           tabla booleana correspondiente.
```

2

	A	B	C
0	False	True	False
1	False	True	False

1.1.2 Ordenación

```
[8]: from random import shuffle
rand_matrix = np.random.randint(20,size=(5,4))
indices = list(range(5))
shuffle(indices) # mezcla indices
frame = pd.DataFrame(rand_matrix , columns=list('DACB'), index=indices)
display(frame)

# Línea 1: importa la función shuffle del módulo estándar random, que mezcla
↳ una lista in place.
# Línea 2: genera una matriz 5x4 de enteros aleatorios en [0,20).
# Línea 3: crea una lista de índices ordenados [0,1,2,3,4].
# Línea 4: shuffle mezcla esa lista de índices en un orden aleatorio (modifica
↳ 'indices' in place).
# Línea 5: crea el DataFrame usando columnas en el orden 'D','A','C','B'
↳ (deliberadamente
#           desordenado) y usando la lista de índices ya mezclada como etiquetas
↳ de fila.
# Línea 6: muestra la tabla resultante.
# Resultado esperado: una tabla 5x4 con columnas en el orden D,A,C,B y filas
↳ cuyo índice está en
#           un orden aleatorio (no necesariamente 0,1,2,3,4 de forma secuencial).
```

	D	A	C	B
3	14	4	13	7
4	13	9	0	16
2	13	19	7	5
1	2	6	7	18
0	9	17	19	15

```
[9]: # ordenar por índice
display(frame.sort_index(ascending=False))
display(frame)
```

```
# Línea 1: comentario descriptivo.
# Línea 2: sort_index(ascending=False) devuelve una NUEVA tabla con las filas
↳reordenadas según
#           su etiqueta de índice, de mayor a menor (no modifica 'frame'
↳original).
# Línea 3: muestra 'frame' de nuevo, confirmando que sigue en su orden original
↳(sort_index no
#           fue in place).
# Resultado esperado: primero la tabla con las filas ordenadas por índice
↳descendente, luego la
#           tabla original sin cambios en el orden de sus filas.
```

	D	A	C	B
4	13	9	0	16
3	14	4	13	7
2	13	19	7	5
1	2	6	7	18
0	9	17	19	15

	D	A	C	B
3	14	4	13	7
4	13	9	0	16
2	13	19	7	5
1	2	6	7	18
0	9	17	19	15

```
[10]: #ordenar por columna
display(frame.sort_index(axis=1, ascending=False))

# Línea 1: comentario descriptivo.
# Línea 2: sort_index(axis=1, ...) ordena por el eje de las COLUMNAS (axis=1)
↳en lugar de las
#           filas, reordenando alfabéticamente de forma descendente los nombres
↳de columna
#           (D,C,B,A).
# Resultado esperado: la misma tabla pero con las columnas reordenadas de la D
↳a la A.
```

	D	C	B	A
3	14	13	7	4
4	13	0	16	9
2	13	7	5	19
1	2	7	18	6
0	9	19	15	17

```
[11]: # ordenar filas por valor en columna
display(frame.sort_values(by='A', ascending=True))
```

```
# Línea 1: comentario descriptivo.
# Línea 2: sort_values(by='A', ascending=True) reordena las FILAS del DataFrame
↳según los
#     valores de la columna 'A', de menor a mayor.
# Resultado esperado: la tabla con las filas reordenadas de forma que la
↳columna A quede en
#     orden ascendente.
```

	D	A	C	B
3	14	4	13	7
1	2	6	7	18
4	13	9	0	16
0	9	17	19	15
2	13	19	7	5

```
[12]: # ordenar columnas por valor en fila
display(frame.sort_values(by=1, axis=1, ascending=True))

# Línea 1: comentario descriptivo.
# Línea 2: sort_values(by=1, axis=1, ...) ordena las COLUMNAS (axis=1) tomando
↳como referencia
#     los valores de la FILA con etiqueta de índice 1, de menor a mayor.
# Resultado esperado: la tabla con las columnas reordenadas según el valor que
↳cada una tiene
#     en la fila de índice 1 (nota: requiere que exista una fila con esa
↳etiqueta exacta,
#     lo cual depende del orden aleatorio de índices generado antes).
```

	D	A	C	B
3	14	4	13	7
4	13	9	0	16
2	13	19	7	5
1	2	6	7	18
0	9	17	19	15

```
[13]: # ordenar por valor en columna y guardar cambios
frame.sort_values(by='A', ascending=False, inplace=True)
display(frame)

# Línea 1: comentario descriptivo.
# Línea 2: inplace=True hace que sort_values MODIFIQUE 'frame' directamente (de
↳mayor a menor
#     según la columna 'A'), sin necesidad de reasignarlo.
# Línea 3: muestra 'frame', que esta vez SÍ queda reordenado de forma
↳permanente.
```

```
# Resultado esperado: la tabla con las filas ya reordenadas de forma
↳descendente por la
#      columna A, y este orden se mantiene para las celdas siguientes.
```

	D	A	C	B
2	13	19	7	5
0	9	17	19	15
4	13	9	0	16
1	2	6	7	18
3	14	4	13	7

1.1.3 Ranking

- Construir un ranking de valores.

```
[14]: display(frame)

# Línea 1: muestra el DataFrame en su estado actual (ya ordenado
↳descendentemente por 'A' desde
#      la celda anterior), como punto de referencia antes de calcular el
↳ranking.
# Resultado esperado: la tabla 5x4 ordenada por la columna A de mayor a menor.
```

	D	A	C	B
2	13	19	7	5
0	9	17	19	15
4	13	9	0	16
1	2	6	7	18
3	14	4	13	7

```
[15]: display(frame.rank(method='max', axis=1))

# Línea 1: rank() asigna a cada valor su posición relativa (ranking) dentro del
↳eje indicado;
#      axis=1 calcula el ranking POR FILA (comparando los valores de
↳D,A,C,B entre sí en
#      cada fila); method='max' indica que, en caso de empate entre valores
↳iguales, se les
#      asigna a todos el ranking MÁS ALTO posible entre los empatados.
# Resultado esperado: una tabla del mismo tamaño donde cada celda contiene un
↳número del 1 al 4
#      indicando la posición relativa de ese valor dentro de su fila
↳(1=menor, 4=mayor).
```

	D	A	C	B
2	3.0	4.0	2.0	1.0
0	1.0	3.0	4.0	2.0
4	3.0	2.0	1.0	4.0

```
1  1.0  2.0  3.0  4.0
3  4.0  1.0  3.0  2.0
```

```
[16]: # Imprimir, uno a uno, los valores de la columna 'C' de mayor a menor
for x in frame.sort_values(by='C', ascending=False)['C'].values:
    print(x)

# Línea 1: comentario descriptivo.
# Línea 2: primero ordena todo el DataFrame por la columna 'C' de mayor a menor
#           (sort_values(by='C', ascending=False)), luego selecciona solo esa
↳ columna
#           (['C']), y finalmente .values la convierte en un ndarray para iterar
↳ sobre ella.
# Línea 3: imprime cada valor de la columna C, ya en orden descendente, uno por
↳ línea.
# Resultado esperado: 5 líneas, cada una con un valor de la columna C,
↳ ordenados de mayor a
#           menor (valores concretos dependen de la matriz aleatoria generada).
```

```
19
13
7
7
0
```

1.2 2. Operaciones

1.2.1 Operaciones matemáticas entre objetos

```
[17]: matrixA = np.random.randint(100,size=(4,4))
matrixB = np.random.randint(100,size=(4,4))
frameA = pd.DataFrame(matrixA)
frameB = pd.DataFrame(matrixB)
display(frameA)
display(frameB)

# Línea 1-2: generan dos matrices 4x4 de enteros aleatorios en [0,100),
↳ independientes entre sí.
# Línea 3-4: convierten cada matriz en un DataFrame (con índices y columnas
↳ numéricos por defecto).
# Línea 5-6: muestran ambas tablas.
# Resultado esperado: dos tablas 4x4 con enteros aleatorios entre 0 y 99,
↳ distintas entre sí.
```

```
      0   1   2   3
0   4  22  34   9
1   1  78  31  42
2  39  34  12  91
3  79   4  61  87
```


	0	1	2	3
0	77	59	33	31
1	7	92	65	88
2	61	53	75	26
3	33	14	20	38

```
[18]: # a través de métodos u operadores
display(frameA + frameB == frameA.add(frameB))
display(frameA + frameB)

# Línea 1: comentario descriptivo.
# Línea 2: compara, elemento a elemento, si sumar con el operador '+' da el
↪ MISMO resultado que
#      usar el método explícito .add(); ambas formas son equivalentes en
↪ pandas.
# Línea 3: muestra el resultado de la suma (con el operador '+') entre ambos
↪ DataFrames, que se
#      realiza elemento a elemento al tener ambos la misma forma e índices/
↪ columnas.
# Resultado esperado: una tabla de puros True (confirmando que '+' y .add() dan
↪ resultados
#      idénticos), seguida de la tabla 4x4 con la suma elemento a elemento
↪ de frameA y frameB.
```

	0	1	2	3
0	True	True	True	True
1	True	True	True	True
2	True	True	True	True
3	True	True	True	True

	0	1	2	3
0	81	81	67	40
1	8	170	96	130
2	100	87	87	117
3	112	18	81	125

```
[19]: display(frameB - frameA == frameB.sub(frameA))
display(frameB - frameA)

# Línea 1: compara si restar con '-' da el mismo resultado que el método .sub().
# Línea 2: muestra el resultado de la resta elemento a elemento (frameB -
↪ frameA).
# Resultado esperado: una tabla de puros True, seguida de la tabla 4x4 con la
↪ diferencia
#      elemento a elemento entre ambos DataFrames.
```

	0	1	2	3
0	True	True	True	True
1	True	True	True	True

```

2 True True True True
3 True True True True

   0  1  2  3
0 73 37 -1 22
1  6 14 34 46
2 22 19 63 -65
3 -46 10 -41 -49

```

```

[20]: # si los frames no son iguales, valor por defecto NaN
frameC = pd.DataFrame(np.random.randint(100,size=(3,3)))
display(frameA)
display(frameC)
display(frameC + frameA)

# Línea 1: comentario descriptivo.
# Línea 2: crea un tercer DataFrame frameC de tamaño 3x3 (más pequeño que
↪ frameA, que es 4x4).
# Línea 3-4: muestra frameA (4x4) y frameC (3x3).
# Línea 5: al sumar dos DataFrames con distinta forma, pandas ALINEA por índice/
↪ columna:
#           donde ambos coinciden (la intersección 3x3) suma normalmente; donde
↪ solo existe en
#           uno de los dos (la fila y columna extra de frameA, índice/columna
↪ 3), el resultado
#           es NaN por defecto.
# Resultado esperado: la suma da una tabla 4x4 donde la submatriz 3x3 superior
↪ izquierda tiene
#           valores numéricos sumados, y toda la última fila y última columna
↪ quedan en NaN.

```

```

   0  1  2  3
0  4 22 34  9
1  1 78 31 42
2 39 34 12 91
3 79  4 61 87

```

```

   0  1  2
0 54 33 58
1  6 49 50
2 68 93 67

```

```

   0  1  2  3
0 58.0 55.0 92.0 NaN
1  7.0 127.0 81.0 NaN
2 107.0 127.0 79.0 NaN
3  NaN  NaN  NaN NaN

```

```
[21]: # se puede especificar el valor por defecto con el argumento fill_value
display(frameA.add(frameC, fill_value=0))

# Línea 1: comentario descriptivo.
# Línea 2: fill_value=0 indica que, para las posiciones donde falte un valor en
↳ alguno de los
#         dos DataFrames (por no coincidir en forma), se use 0 en lugar de NaN
↳ antes de sumar;
#         así la última fila/columna de frameA se suma con 0 en vez de quedar
↳ en NaN.
# Resultado esperado: una tabla 4x4 completamente numérica (sin NaN), donde la
↳ fila y columna
#         extra de frameA conservan sus valores originales (al sumarles 0).
```

	0	1	2	3
0	58.0	55.0	92.0	9.0
1	7.0	127.0	81.0	42.0
2	107.0	127.0	79.0	91.0
3	79.0	4.0	61.0	87.0

Operadores aritméticos solo válidos en elementos aceptables

```
[22]: frameD = pd.DataFrame({0: ['a', 'b'], 1: ['d', 'f']})
display(frameD)
frameA - frameD

# Línea 1: crea un DataFrame de 2x2 con valores de tipo texto (strings).
# Línea 2: lo muestra.
# Línea 3: intentar restar frameA (numérico) menos frameD (texto) falla, porque
↳ la resta
#         aritmética no está definida para strings; al ser la última expresión
↳ de la celda, el
#         error se muestra como resultado de la ejecución.
# Resultado esperado: TypeError (o similar), indicando que la operación de
↳ resta no puede
#         aplicarse entre valores numéricos y de texto.
```

	0	1
0	a	d
1	b	f

```
-----
TypeError                                Traceback (most recent call last)
Cell In[22], line 3
      1 frameD = pd.DataFrame({0: ['a', 'b'], 1: ['d', 'f']})
      2 display(frameD)
----> 3 frameA - frameD
      4
```

```

5 # Línea 1: crea un DataFrame de 2x2 con valores de tipo texto (strings)
6 # Línea 2: lo muestra.

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\ops\common.
↳py:85, in _unpack_zerodim_and_defer.<locals>.new_method(self, other)
    82     other = sanitize_array(other, None)
    83     other = ensure_wrapped_if_datetimelike(other)
--> 85 return method(self, other)

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\arraylike.p :
↳198, in OpsMixin.__sub__(self, other)
    196 @unpack_zerodim_and_defer("__sub__")
    197 def __sub__(self, other):
--> 198     return self._arith_method(other, operator.sub)

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\frame.py:
↳9143, in DataFrame._arith_method(self, other, op)
    9141     def _arith_method(self, other, op):
    9142         if self._should_reindex_frame_op(other, op, 1, None, None):
-> 9143             return self._arith_method_with_reindex(other, op)
    9144
    9145         axis: Literal[1] = 1 # only relevant for Series other case
    9146         other = ops.maybe_prepare_scalar_for_op(other, (self.
↳shape[axis],))

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\frame.py:
↳9279, in DataFrame._arith_method_with_reindex(self, right, op)
    9275         new_right = new_right.copy(deep=False)
    9276         new_left.columns = cols
    9277         new_right.columns = cols
    9278
-> 9279         result = op(new_left, new_right)
    9280
    9281         # Do the join on the columns instead of using left._align_for_o
    9282         # to avoid constructing two potentially large/sparse DataFrame

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\ops\common.
↳py:85, in _unpack_zerodim_and_defer.<locals>.new_method(self, other)
    82     other = sanitize_array(other, None)
    83     other = ensure_wrapped_if_datetimelike(other)
--> 85 return method(self, other)

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\arraylike.p :
↳198, in OpsMixin.__sub__(self, other)
    196 @unpack_zerodim_and_defer("__sub__")
    197 def __sub__(self, other):
--> 198     return self._arith_method(other, operator.sub)

```

```
File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\frame.py:
```

```
→9151, in DataFrame._arith_method(self, other, op)
    9147
    9148         self, other = self._align_for_op(other, axis, flex=True,
→level=None)
    9149
    9150         with np.errstate(all="ignore"):
-> 9151             new_data = self._dispatch_frame_op(other, op, axis=axis)
    9152         return self._construct_result(new_data, other=other)
```

```
File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\frame.py:
```

```
→9194, in DataFrame._dispatch_frame_op(self, right, func, axis)
    9190         # fails in cases with empty columns reached via
    9191         # _frame_arith_method_with_reindex
    9192
    9193         # TODO operate_blockwise expects a manager of the same type
-> 9194         bm = self._mgr.operate_blockwise(
    9195             right._mgr,
    9196             array_op,
    9197         )
```

```
File
```

```
→~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\internals\managers.
→py:1691, in BlockManager.operate_blockwise(self, other, array_op)
    1687 def operate_blockwise(self, other: BlockManager, array_op) ->
→BlockManager:
    1688     """
    1689     Apply array_op blockwise with another (aligned) BlockManager.
    1690     """
-> 1691     return operate_blockwise(self, other, array_op)
```

```
File
```

```
→~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\internals\ops.
→py:65, in operate_blockwise(left, right, array_op)
    63 res_blks: list[Block] = []
    64 for lvals, rvals, locs, left_ea, right_ea, rblk in
→_iter_block_pairs(left, right):
---> 65     res_values = array_op(lvals, rvals)
    66     if (
    67         left_ea
    68         and not right_ea
    69         and hasattr(res_values, "reshape")
    70         and not is_1d_only_ea_dtype(res_values.dtype)
    71     ):
    72         res_values = res_values.reshape(1, -1)
```

```

File
↳ ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\ops\array_ops.
↳ py:279, in arithmetic_op(left, right, op)
    270 right = ensure_wrapped_if_datetimelike(right)
    272 if (
    273     should_extension_dispatch(left, right)
    274     or isinstance(right, (Timedelta, BaseOffset, Timestamp))
    (...) 277     # Timedelta/Timestamp and other custom scalars are included i
↳ the check
    278     # because numexpr will fail on it, see GH#31457
--> 279     res_values = op(left, right)
    280 else:
    281     # TODO we should handle EAs consistently and move this check before
↳ the if/else
    282     # (https://github.com/pandas-dev/pandas/issues/41165)
    283     # error: Argument 2 to "_bool_arith_check" has incompatible type
    284     # "Union[ExtensionArray, ndarray[Any, Any]]"; expected "ndarray[Any
↳ Any]"
    285     _bool_arith_check(op, left, right) # type: ignore[arg-type]

```

```

File
↳ ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\arrays\numpy_.
↳ py:212, in NumpyExtensionArray.__array_ufunc__(self, ufunc, method, *inputs,
↳ **kwargs)
    205 def __array_ufunc__(self, ufunc: np.ufunc, method: str, *inputs,
↳ **kwargs):
    206     # Lightly modified version of
    207     # https://numpy.org/doc/stable/reference/generated/numpy.lib.mixins
↳ NDArraryOperatorsMixin.html
    208     # The primary modification is not boxing scalar return values
    209     # in NumpyExtensionArray, since pandas' ExtensionArrays are 1-d.
    210     out = kwargs.get("out", ())
--> 212     result = arraylike.maybe_dispatch_ufunc_to_dunder_op(
    213         self, ufunc, method, *inputs, **kwargs
    214     )
    215     if result is not NotImplemented:
    216         return result

```

```

File pandas/_libs/ops_dispatch.pyx:113, in pandas._libs.ops_dispatch.
↳ maybe_dispatch_ufunc_to_dunder_op()
--> 113 'Could not get source, probably due dynamically evaluated source code.'

```

```

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\ops\common.
↳ py:85, in _unpack_zerodim_and_defer.<locals>.new_method(self, other)
    82     other = sanitize_array(other, None)
    83     other = ensure_wrapped_if_datetimelike(other)
--> 85 return method(self, other)

```

```

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\arraylike.p :
  ↪202, in OpsMixin._rsub__(self, other)
    200 @unpack_zerodim_and_defer("__rsub__")
    201 def __rsub__(self, other):
--> 202     return self._arith_method(other, roperator.rsub)

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\arrays\string_
py:1218, in StringArray._cmp_method(self, other, op)
    1216 result = np.empty_like(self._ndarray, dtype="object")
    1217 result[mask] = self.dtype.na_value
-> 1218 result[valid] = op(self._ndarray[valid], other)
    1219 if isinstance(other, Path):
    1220     # GH#61940
    1221     return result

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\roperator.p :
  ↪16, in rsub(left, right)
    15 def rsub(left, right):
---> 16     return right - left

TypeError: unsupported operand type(s) for -: 'int' and 'str'

```

1.2.2 Operaciones entre Series y DataFrames

```

[ ]: rand_matrix = np.random.randint(10, size=(3, 4))
df = pd.DataFrame(rand_matrix , columns=list('ABCD'))
display(df)
display(df.iloc[0])
display(type(df.iloc[0]))
# uso común, averiguar la diferencia entre una fila y el resto
display(df - df.iloc[0])
display(df.sub(df.iloc[0], axis=1))
# Por columnas cómo se restaría
display(df.sub(df['A'], axis=0))

# Línea 1: genera una matriz 3x4 de enteros aleatorios en [0,10).
# Línea 2: crea el DataFrame con columnas A,B,C,D.
# Línea 3: lo muestra.
# Línea 4: df.iloc[0] extrae la primera fila como una Series (con las columnas
  ↪A,B,C,D como
#
  ↪índice de esa Series).
# Línea 5: muestra esa Series.
# Línea 6: confirma que el tipo de df.iloc[0] es una Series de pandas.
# Línea 7: comentario descriptivo sobre el uso común de esta operación.
# Línea 8: al restar un DataFrame menos una Series (df - df.iloc[0]), pandas
  ↪alinea

```

```

#             automáticamente por ETIQUETA DE COLUMNA (broadcasting fila a fila):
↳ resta esa
#             fila/Series a CADA fila del DataFrame.
# Línea 9: df.sub(df.iloc[0], axis=1) es la forma explícita equivalente,
↳ indicando axis=1 para
#             alinear por columnas (mismo resultado que la resta con '-').
# Línea 10: comentario descriptivo sobre cómo hacerlo por columnas en vez de
↳ por filas.
# Línea 11: df.sub(df['A'], axis=0) resta la columna 'A' (una Series indexada
↳ por fila) a CADA
#             columna del DataFrame, alineando por índice de FILA (axis=0) en
↳ lugar de por columna.
# Resultado esperado: la tabla original 3x4; la primera fila como Series; el
↳ tipo Series; una
#             tabla donde cada fila muestra su diferencia respecto a la primera
↳ fila (la primera
#             fila queda en ceros); el mismo resultado con .sub(); y una tabla
↳ donde cada columna
#             muestra su diferencia respecto a la columna 'A' (la columna A queda
↳ en ceros).

```

pandas se basa en NumPy: los operadores binarios y unarios de NumPy son aceptables.

Tabla de referencia con operadores unarios (abs, sqrt, exp, log, sign, ceil, floor, isnan, funciones trigonométricas) y binarios (add, subtract, multiply, divide, maximum, minimum, equal, greater, etc.), análoga a la vista en NumPy.

Aplicación de funciones a medida con lambda

```

[23]: rand_matrix = np.random.randint(10, size=(3, 4))
frame = pd.DataFrame(rand_matrix , columns=list('ABCD'))
display(frame)
print(frame.apply(lambda x : x.max() - x.min(), axis = 1)) # diferencia por
↳ columna

# Línea 1: genera una matriz 3x4 de enteros aleatorios en [0,10).
# Línea 2: crea el DataFrame con columnas A,B,C,D.
# Línea 3: lo muestra.
# Línea 4: apply aplica la función lambda a cada elemento del eje indicado;
↳ axis=1 significa que
#             la función se aplica a CADA FILA (tomada como una Series x),
↳ calculando su rango
#             (máximo menos mínimo). A pesar de que el comentario original dice
↳ "por columna", con
#             axis=1 realmente se calcula UN valor POR FILA (posible error de
↳ redacción en el
#             material original, ya que axis=1 opera fila a fila, no columna a
↳ columna).

```



```
# Resultado esperado: una Series con 3 valores (uno por fila), cada uno igual al rango
# (máximo - mínimo) de esa fila.
```

```
   A  B  C  D
0  8  7  7  0
1  0  7  8  3
2  0  6  7  0
```

```
0    8
1    8
2    7
```

```
dtype: int32
```

```
[24]: def max_min(x):
        return x.max() - x.min()

print(frame.apply(max_min, axis = 0)) # diferencia por columna

# Línea 1-2: define una función nombrada equivalente a la lambda anterior, que recibe una
# Series x y devuelve su rango (max - min); es más legible y reutilizable que una lambda.
# Línea 3: aplica esa función con axis=0, lo que significa que se aplica a CADA COLUMNA (tomada
# como una Series x) de forma independiente.
# Resultado esperado: una Series con 4 valores (uno por columna A,B,C,D), cada uno igual al
# rango de esa columna.
```

```
A    8
B    1
C    1
D    3
```

```
dtype: int32
```

```
[25]: # diferencia entre min y max por fila (no columna)
rand_matrix = np.random.randint(10, size=(3, 4))
frame = pd.DataFrame(rand_matrix , columns=list('ABCD'))
display(frame)
print(frame.apply(lambda x : x.max() - x.min(), axis = 1)) # diferencia por fila

# Línea 1: comentario descriptivo aclarando (correctamente esta vez) que axis=1 calcula por fila.
# Línea 2: genera una nueva matriz 3x4 de enteros aleatorios en [0,10).
# Línea 3: crea el DataFrame.
# Línea 4: lo muestra.
```

```
# Línea 5: aplica la lambda con axis=1, calculando el rango (max - min) de cada
↪FILA.
# Resultado esperado: una tabla 3x4 con nuevos valores aleatorios, seguida de
↪una Series con 3
#           valores (uno por fila), cada uno igual al rango de esa fila.
```

```
      A  B  C  D
0  2  7  7  5
1  7  5  2  9
2  2  1  9  0

0      5
1      7
2      9
dtype: int32
```

1.3 3. Estadística descriptiva

- Análisis preliminar de los datos.
- Para Series y DataFrame.

Funciones: count, describe, min/max, argmin/argmax, idxmin/idxmax, sum, mean, median, mad, var, std, cumsum, diff.

```
[26]: diccionario = { "nombre" : ["Marisa","Laura","Manuel", "Carlos"], "edad" :
↪[34,34,11, 30],
                "puntos" : [98,12,98,np.nan], "genero": ["F", "F", "M", "M"] }
frame = pd.DataFrame(diccionario)
display(frame)
display(frame.describe()) # datos generales de elementos

# Línea 1-2: define un diccionario con 4 columnas: 'nombre' (texto), 'edad'
↪(entero), 'puntos'
#           (float, incluye un valor NaN para Carlos) y 'genero' (texto).
# Línea 3: crea el DataFrame a partir del diccionario.
# Línea 4: lo muestra.
# Línea 5: describe() calcula estadísticas (count, mean, std, min, cuartiles,
↪max) SOLO para las
#           columnas numéricas por defecto ('edad' y 'puntos'); 'puntos' tendrá
↪count=3 porque
#           ignora el NaN.
# Resultado esperado: tabla de 4 filas x 4 columnas con los datos de las
↪personas, seguida de
#           una tabla de estadísticas descriptivas para 'edad' y 'puntos'.
```

```
      nombre  edad  puntos  genero
0  Marisa      34    98.0        F
1   Laura      34    12.0        F
2  Manuel      11    98.0        M
```

```
3 Carlos 30 NaN M
```

```
      edad  puntos
count  4.000000  3.000000
mean   27.250000  69.333333
std    10.996211  49.652123
min    11.000000  12.000000
25%    25.250000  55.000000
50%    32.000000  98.000000
75%    34.000000  98.000000
max     34.000000  98.000000
```

```
[27]: # operadores básicos
print(frame.sum())
display(frame)
print(frame.sum(axis=1, numeric_only=True))

# Línea 1: comentario descriptivo.
# Línea 2: sum() sin axis suma por COLUMNA (axis=0 por defecto); para columnas
# de texto ('nombre',
# 'genero') la "suma" concatena todos los strings de esa columna; para
# las numéricas
# suma sus valores (ignorando NaN).
# Línea 3: muestra el DataFrame de nuevo como referencia.
# Línea 4: sum(axis=1, numeric_only=True) suma POR FILA, y numeric_only=True
# excluye las
# columnas de texto para evitar errores/concatenaciones al sumar
# horizontalmente.
# Resultado esperado: primero una Series con la suma por columna (texto
# concatenado para
# 'nombre' y 'genero', números sumados para 'edad' y 'puntos'), la
# tabla original, y
# finalmente una Series con 4 valores (uno por persona) sumando 'edad'
# + 'puntos' de
# cada fila (con NaN tratado como 0 en la suma de Carlos).
```

```
nombre  MarisaLauraManuelCarlos
edad                109
puntos                208.0
genero                FFMM
dtype: object
```

```
      nombre  edad  puntos  genero
0  Marisa    34    98.0      F
1  Laura    34    12.0      F
2  Manuel   11    98.0      M
3  Carlos   30     NaN      M
```

```
0    132.0
```

```
1      46.0
2     109.0
3      30.0
dtype: float64
```

```
[28]: frame.mean(numeric_only=True)

# Línea 1: mean(numeric_only=True) calcula la media de cada columna NUMÉRICA
#   ↳('edad' y
#   ↳'puntos'), ignorando automáticamente los valores NaN en el cálculo;
#   ↳al ser la última
#   ↳expresión de la celda se muestra el resultado.
# Resultado esperado: una Series con la media de 'edad' (aprox. 27.25) y la
#   ↳media de 'puntos'
#   ↳(calculada solo sobre los 3 valores no nulos: (98+12+98)/3 69.33).
```

```
[28]: edad      27.250000
      puntos    69.333333
      dtype: float64
```

```
[29]: frame.cumsum()

# Línea 1: cumsum() calcula la suma acumulada fila a fila para cada columna;
#   ↳para columnas de
#   ↳texto, acumula concatenando los strings progresivamente; para las
#   ↳numéricas, va
#   ↳sumando los valores anteriores (los NaN se propagan como NaN en las
#   ↳filas siguientes
#   ↳de esa columna a partir de donde aparecen, según el comportamiento
#   ↳por defecto).
# Resultado esperado: una tabla del mismo tamaño que 'frame', donde cada celda
#   ↳contiene la
#   ↳acumulación de los valores de esa columna hasta esa fila.
```

```
[29]:
```

	nombre	edad	puntos	genero
0	Marisa	34	98.0	F
1	MarisaLaura	68	110.0	FF
2	MarisaLauraManuel	79	208.0	FFM
3	MarisaLauraManuelCarlos	109	NaN	FFMM

```
[30]: frame.count(axis=1)

# Línea 1: count(axis=1) cuenta, para cada FILA, cuántos valores NO son NaN
#   ↳entre todas las
#   ↳columnas (incluye columnas de texto, que nunca son NaN en este caso).
# Resultado esperado: una Series con 4 valores; Marisa, Laura y Manuel tendrán
#   ↳4 (todas sus
```

```
#          columnas tienen datos), y Carlos tendrá 3 (porque su columna
↳ 'puntos' es NaN).
```

```
[30]: 0    4
      1    4
      2    4
      3    3
      dtype: int64
```

```
[31]: print(frame['edad'].std())
```

```
# Línea 1: selecciona la columna 'edad' como Series y calcula su desviación
↳ estándar muestral
#          (std, que por defecto usa n-1 en el denominador).
# Resultado esperado: un número float, por ejemplo algo cercano a 10.8 (el
↳ valor exacto depende
#          de los datos: edades 34,34,11,30).
```

```
10.996211468804457
```

```
[32]: frame['edad'].idxmax()
```

```
# Línea 1: idxmax() sobre la columna 'edad' devuelve la ETIQUETA DE ÍNDICE
↳ (posición en el
#          índice del DataFrame, no el valor) donde se encuentra el valor
↳ máximo de esa columna.
# Resultado esperado: 0 (o 1, ya que Marisa y Laura empatan con edad=34; idxmax
↳ devuelve la
#          PRIMERA ocurrencia del máximo, que corresponde a Marisa, índice 0).
```

```
[32]: 0
```

```
[33]: frame['puntos'].idxmin()
```

```
# Línea 1: idxmin() sobre la columna 'puntos' devuelve la etiqueta de índice
↳ donde está el
#          valor mínimo de esa columna, ignorando los NaN en la comparación.
# Resultado esperado: 1 (Laura, con puntos=12, el valor más bajo no nulo).
```

```
[33]: 1
```

```
[34]: # frame con las filas con los valores maximos de una columna
```

```
print(frame['puntos'].max())
display(frame[frame['puntos'] == frame['puntos'].max()])
```

```
# Línea 1: comentario descriptivo.
# Línea 2: max() sobre la columna 'puntos' devuelve el valor máximo (ignorando
↳ NaN).
```

```
# Línea 3: filtra el DataFrame completo para mostrar solo las filas cuyo valor
↳ en 'puntos' es
#          igual a ese máximo (puede haber empates, como en este caso).
# Resultado esperado: 98.0 impreso, seguido de una tabla con las filas de
↳ Marisa y Manuel
#          (ambos con puntos=98, el valor máximo).
```

98.0

	nombre	edad	puntos	genero
0	Marisa	34	98.0	F
2	Manuel	11	98.0	M

```
[35]: frame["ranking"] = frame["puntos"].rank(method='max')

# Línea 1: rank(method='max') calcula el ranking de los valores de 'puntos'
↳ (los NaN reciben
#          NaN como ranking por defecto); en caso de empate, method='max'
↳ asigna a ambos
#          empatados el ranking más alto posible entre ellos; el resultado se
↳ guarda como una
#          NUEVA columna 'ranking' en el DataFrame.
# Resultado esperado: no hay salida impresa directamente; 'frame' queda con una
↳ columna
#          adicional 'ranking' (Marisa y Manuel compartirán el ranking más
↳ alto, 3.0; Laura
#          tendrá 1.0; Carlos tendrá NaN).
```

```
[36]: display(frame)

# Línea 1: muestra el DataFrame final, ya con la columna 'ranking' añadida en
↳ la celda anterior.
# Resultado esperado: tabla de 4 filas x 5 columnas (nombre, edad, puntos,
↳ genero, ranking).
```

	nombre	edad	puntos	genero	ranking
0	Marisa	34	98.0	F	3.0
1	Laura	34	12.0	F	1.0
2	Manuel	11	98.0	M	3.0
3	Carlos	30	NaN	M	NaN

1.3.1 3.1 Agregaciones

```
[37]: display(frame)
df = frame.groupby('genero').count()
display(df)

# Línea 1: muestra 'frame' como referencia antes de agrupar.
```

```
# Línea 2: groupby('genero') agrupa las filas según el valor de 'genero' ('F' o
↳ 'M');
#          count() cuenta, para cada grupo, cuántos valores NO nulos hay en
↳ cada una de las
#          demás columnas.
# Línea 3: muestra el resultado agrupado.
# Resultado esperado: una tabla con 2 filas (F, M) y columnas nombre/edad/
↳ puntos/ranking,
#          donde cada celda indica cuántos valores no nulos hay en ese grupo
↳ para esa columna
#          (por ejemplo, 'puntos' para el grupo M será 1, porque Carlos tiene
↳ NaN).
```

	nombre	edad	puntos	genero	ranking
0	Marisa	34	98.0	F	3.0
1	Laura	34	12.0	F	1.0
2	Manuel	11	98.0	M	3.0
3	Carlos	30	NaN	M	NaN

	nombre	edad	puntos	ranking
genero				
F	2	2	2	2
M	2	2	1	1

```
[38]: # si es Nan descarta la fila
df = frame.groupby('puntos').count()
display(df)

# Línea 1: comentario descriptivo.
# Línea 2: groupby('puntos') agrupa por el valor de la columna 'puntos'; por
↳ defecto, groupby
#          DESCARTA las filas cuyo valor de agrupación es NaN (la fila de
↳ Carlos no aparecerá
#          en ningún grupo).
# Línea 3: muestra el resultado agrupado.
# Resultado esperado: una tabla con una fila por cada valor distinto de
↳ 'puntos' presente (12.0
#          y 98.0), mostrando el conteo de valores no nulos de las demás
↳ columnas en cada grupo;
#          Carlos (puntos=NaN) queda excluido por completo.
```

	nombre	edad	genero	ranking
puntos				
12.0	1	1	1	1
98.0	2	2	2	2

```
[39]: display(frame.groupby('genero').mean(numeric_only=True))
```

```
# Línea 1: agrupa por 'genero' y calcula la media de las columnas numéricas
↳('edad', 'puntos',
#
# 'ranking') para cada grupo, ignorando NaN en el cálculo.
# Resultado esperado: una tabla con 2 filas (F, M), mostrando la media de edad,
↳puntos y
#
# ranking para el grupo de mujeres y para el de hombres por separado.
```

	edad	puntos	ranking
genero			
F	34.0	55.0	2.0
M	20.5	98.0	3.0

```
[40]: display(frame.groupby('genero').max())
```

```
# Línea 1: agrupa por 'genero' y calcula el valor MÁXIMO de cada columna
↳(incluyendo columnas
#
# de texto, donde 'máximo' se interpreta como el orden alfabético más
↳alto) para cada
#
# grupo.
# Resultado esperado: una tabla con 2 filas (F, M), mostrando el nombre
↳alfabéticamente mayor,
#
# la edad máxima, los puntos máximos y el ranking máximo de cada grupo.
```

	nombre	edad	puntos	ranking
genero				
F	Marisa	34	98.0	3.0
M	Manuel	30	98.0	3.0

```
[41]: # funciones de agregación de varias columnas para obtener distintos estadísticos
display(frame.groupby('genero')[['edad', 'puntos']].aggregate(['min', 'mean',
↳'max']))
```

```
# Línea 1: comentario descriptivo.
# Línea 2: agrupa por 'genero', selecciona solo las columnas 'edad' y 'puntos',
↳y aggregate
#
# aplica VARIAS funciones estadísticas ('min', 'mean', 'max') a cada
↳una de esas
#
# columnas simultáneamente para cada grupo.
# Resultado esperado: una tabla con columnas jerárquicas (edad/puntos, cada una
↳con min, mean,
#
# max) y 2 filas (F, M), mostrando esos tres estadísticos por grupo y
↳por columna.
```

	edad			puntos		
	min	mean	max	min	mean	max
genero						
F	34	34.0	34	12.0	55.0	98.0
M	11	20.5	30	98.0	98.0	98.0


```
[42]: # Filtrado de los datos en el que el conjunto no supera una media determinada
def media(x):
    return x["edad"].mean() > 30

display(frame)
frame.groupby('genero').filter(media)

# Línea 1: comentario descriptivo.
# Línea 2-3: define una función que recibe un grupo (subconjunto del DataFrame)
# y devuelve True
# si la media de 'edad' de ese grupo es mayor a 30.
# Línea 4: muestra 'frame' como referencia.
# Línea 5: groupby('genero').filter(media) CONSERVA únicamente las filas de los
# grupos completos
# que cumplen la condición (grupos cuya media de edad supera 30),
# descartando por
# completo los grupos que no la cumplen; al ser la última expresión de
# la celda se
# muestra el resultado.
# Resultado esperado: la tabla original mostrada primero, y luego solo las
# filas del grupo
# 'genero' cuya media de edad sea mayor a 30 (por ejemplo, si el grupo
# F tiene media
#  $(34+34)/2=34 > 30$ , se conservan Marisa y Laura; si el grupo M tiene
# media  $(11+30)/2=20.5$ ,
# no supera 30 y se descartan Manuel y Carlos).
```

	nombre	edad	puntos	genero	ranking
0	Marisa	34	98.0	F	3.0
1	Laura	34	12.0	F	1.0
2	Manuel	11	98.0	M	3.0
3	Carlos	30	NaN	M	NaN

```
[42]: nombre edad puntos genero ranking
0 Marisa 34 98.0 F 3.0
1 Laura 34 12.0 F 1.0
```

1.3.2 3.2 Correlaciones

pandas incluye métodos para analizar correlaciones:

- Relación matemática entre dos variables (-1 negativamente relacionadas, 1 positivamente relacionadas, 0 sin relación).
- `obj.corr(obj2)` → medida de correlación entre los datos de ambos objetos.
- Más información: <https://blogs.oracle.com/ai-and-datascience/post/introduction-to-correlation>

3.2.1 Ejemplo Fuel efficiency

- Dataset: <https://archive.ics.uci.edu/ml/datasets/Auto+MPG>

```
[43]: import pandas as pd
path = 'http://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/
↳auto-mpg.data'
mpg_data = pd.read_csv(path, sep='\s+', header=None,
                        names = ['mpg', 'cilindros',
↳'desplazamiento', 'potencia',
                        'peso', 'aceleracion', 'año', 'origen',
↳'nombre'],
                        na_values='?', engine='c')

# Línea 1: importa pandas (por si el kernel se reinició).
# Línea 2: define la URL remota del dataset Auto MPG (eficiencia de combustible
↳de automóviles).
# Línea 3-6: pd.read_csv descarga y parsea el archivo directamente desde la URL;
↳ sep='\s+'
#           indica que las columnas están separadas por uno o más espacios en
↳blanco (no por
#           comas); header=None indica que el archivo no trae fila de cabecera,
↳por lo que se
#           asignan manualmente los 9 nombres de columna con 'names';
↳na_values='?' le dice a
#           pandas que interprete el carácter '?' (usado en este dataset para
↳datos faltantes,
#           típicamente en 'potencia') como NaN; engine='c' usa el motor de
↳parseo en C, más
#           rápido, compatible con el separador de expresión regular \s+.
# Resultado esperado: no hay salida impresa; 'mpg_data' queda cargado en
↳memoria con 398 filas
#           y 9 columnas.
```

```
[44]: display(mpg_data.sample(5))

# Línea 1: sample(5) devuelve 5 filas aleatorias del dataset para una
↳inspección rápida.
# Resultado esperado: una tabla de 5 filas x 9 columnas con datos de autos
↳elegidos al azar
#           (distintos en cada ejecución).
```

	mpg	cilindros	desplazamiento	potencia	peso	aceleracion	año	\
72	15.0	8	304.0	150.0	3892.0	12.5	72	
157	15.0	8	350.0	145.0	4440.0	14.0	75	
258	20.6	6	231.0	105.0	3380.0	15.8	78	
163	18.0	6	225.0	95.0	3785.0	19.0	75	
303	31.8	4	85.0	65.0	2020.0	19.2	79	

	origen	nombre
72	1	amc matador (sw)
157	1	chevrolet bel air
258	1	buick century special
163	1	plymouth fury
303	3	datsum 210

```
[45]: display(mpg_data.describe(include='all'))
```

```
# Línea 1: describe(include='all') calcula estadísticas para TODAS las
↳ columnas, tanto
#           numéricas (count, mean, std, min, cuartiles, max) como de texto
↳ (count, unique, top,
#           freq para 'nombre'), dejando NaN en las combinaciones no aplicables.
# Resultado esperado: una tabla ampliada de estadísticas descriptivas para las
↳ 9 columnas del
#           dataset, incluyendo la columna 'nombre' de tipo texto.
```

	mpg	cilindros	desplazamiento	potencia	peso \
count	398.000000	398.000000	398.000000	392.000000	398.000000
unique	NaN	NaN	NaN	NaN	NaN
top	NaN	NaN	NaN	NaN	NaN
freq	NaN	NaN	NaN	NaN	NaN
mean	23.514573	5.454774	193.425879	104.469388	2970.424623
std	7.815984	1.701004	104.269838	38.491160	846.841774
min	9.000000	3.000000	68.000000	46.000000	1613.000000
25%	17.500000	4.000000	104.250000	75.000000	2223.750000
50%	23.000000	4.000000	148.500000	93.500000	2803.500000
75%	29.000000	8.000000	262.000000	126.000000	3608.000000
max	46.600000	8.000000	455.000000	230.000000	5140.000000

	aceleracion	año	origen	nombre
count	398.000000	398.000000	398.000000	398
unique	NaN	NaN	NaN	305
top	NaN	NaN	NaN	ford pinto
freq	NaN	NaN	NaN	6
mean	15.568090	76.010050	1.572864	NaN
std	2.757689	3.697627	0.802055	NaN
min	8.000000	70.000000	1.000000	NaN
25%	13.825000	73.000000	1.000000	NaN
50%	15.500000	76.000000	1.000000	NaN
75%	17.175000	79.000000	2.000000	NaN
max	24.800000	82.000000	3.000000	NaN

```
[46]: mpg_data.info()
```

```
# Línea 1: info() imprime un resumen del DataFrame: número de filas, columnas,
↳ tipo de dato de
```

```
#          cada una, cantidad de valores no nulos por columna y uso de memoria.
# Resultado esperado: un bloque de texto indicando 398 filas, 9 columnas, y que
  ↳ 'potencia'
#          tiene menos valores no nulos que el resto (debido a los '?')
  ↳ convertidos en NaN).
```

```
<class 'pandas.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   mpg              398 non-null   float64
1   cilindros        398 non-null   int64
2   desplazamiento   398 non-null   float64
3   potencia         392 non-null   float64
4   peso             398 non-null   float64
5   aceleracion      398 non-null   float64
6   año              398 non-null   int64
7   origen           398 non-null   int64
8   nombre           398 non-null   str
dtypes: float64(5), int64(3), str(1)
memory usage: 28.1 KB
```

3.2.2 Correlaciones entre valores

```
[47]: mpg_data['mpg'].corr(mpg_data['peso']) # + mpg = - peso

# Línea 1: corr() calcula el coeficiente de correlación (Pearson por defecto)
  ↳ entre la columna
#          'mpg' (millas por galón, eficiencia) y la columna 'peso' del
  ↳ vehículo; al ser la
#          última expresión de la celda se muestra el resultado; el comentario
  ↳ indica que se
#          espera una relación inversa (a más peso, menos eficiencia).
# Resultado esperado: un número float negativo cercano a -0.83, confirmando una
  ↳ fuerte
#          correlación negativa entre eficiencia y peso.
```

```
[47]: np.float64(-0.8317409332443352)
```

```
[48]: mpg_data['peso'].corr(mpg_data['aceleracion']) # + peso = - aceleracion

# Línea 1: calcula la correlación entre 'peso' y 'aceleracion'; el comentario
  ↳ anticipa que a
#          mayor peso, menor capacidad de aceleración (tiempo de 0 a 60mph más
  ↳ alto se traduce
#          en menor valor de 'aceleracion' en este dataset, según cómo esté
  ↳ definida la columna).
```

```
# Resultado esperado: un número float negativo (correlación negativa moderada/
↳fuerte entre
#      ambas variables).
```

```
[48]: np.float64(-0.4174573199403932)
```

3.2.3 Correlaciones entre todos los valores

```
[49]: mpg_data.corr(numeric_only=True)
```

```
# Línea 1: corr(numeric_only=True) calcula la MATRIZ de correlación entre todas
↳las columnas
#      numéricas del dataset (excluyendo 'nombre', que es texto),
↳comparando cada par de
#      columnas entre sí.
# Resultado esperado: una tabla cuadrada (matriz simétrica) donde cada celda
↳[i,j] contiene el
#      coeficiente de correlación entre la columna i y la columna j; la
↳diagonal siempre
#      vale 1.0 (correlación de una columna consigo misma).
```

```
[49]:
```

	mpg	cilindros	desplazamiento	potencia	peso \
mpg	1.000000	-0.775396	-0.804203	-0.778427	-0.831741
cilindros	-0.775396	1.000000	0.950721	0.842983	0.896017
desplazamiento	-0.804203	0.950721	1.000000	0.897257	0.932824
potencia	-0.778427	0.842983	0.897257	1.000000	0.864538
peso	-0.831741	0.896017	0.932824	0.864538	1.000000
aceleracion	0.420289	-0.505419	-0.543684	-0.689196	-0.417457
año	0.579267	-0.348746	-0.370164	-0.416361	-0.306564
origen	0.563450	-0.562543	-0.609409	-0.455171	-0.581024

	aceleracion	año	origen
mpg	0.420289	0.579267	0.563450
cilindros	-0.505419	-0.348746	-0.562543
desplazamiento	-0.543684	-0.370164	-0.609409
potencia	-0.689196	-0.416361	-0.455171
peso	-0.417457	-0.306564	-0.581024
aceleracion	1.000000	0.288137	0.205873
año	0.288137	1.000000	0.180662
origen	0.205873	0.180662	1.000000

```
[50]: #año y origen no parecen correlacionables
#eliminar columnas de la correlacion
corr_data = mpg_data.drop(['año', 'origen'], axis=1).corr(numeric_only=True)
display(corr_data)

# Línea 1-2: comentarios indicando que 'año' y 'origen' son variables más
↳categóricas/temporales
```

```
# y no aportan una correlación lineal significativa con el resto.
# Línea 3: drop(['año', 'origen'], axis=1) elimina esas dos columnas antes de
↳ calcular la matriz
# de correlación, quedándose con las columnas verdaderamente numéricas
↳ continuas.
# Línea 4: muestra la matriz de correlación resultante, ya sin 'año' ni
↳ 'origen'.
# Resultado esperado: una matriz de correlación más pequeña y enfocada (mpg,
↳ cilindros,
# desplazamiento, potencia, peso, aceleración).
```

	mpg	cilindros	desplazamiento	potencia	peso	\
mpg	1.000000	-0.775396	-0.804203	-0.778427	-0.831741	
cilindros	-0.775396	1.000000	0.950721	0.842983	0.896017	
desplazamiento	-0.804203	0.950721	1.000000	0.897257	0.932824	
potencia	-0.778427	0.842983	0.897257	1.000000	0.864538	
peso	-0.831741	0.896017	0.932824	0.864538	1.000000	
aceleracion	0.420289	-0.505419	-0.543684	-0.689196	-0.417457	

	aceleracion
mpg	0.420289
cilindros	-0.505419
desplazamiento	-0.543684
potencia	-0.689196
peso	-0.417457
aceleracion	1.000000

```
[51]: # representación gráfica matplotlib
import matplotlib.pyplot as plt

# Línea 1: comentario descriptivo.
# Línea 2: importa el módulo pyplot de matplotlib con el alias 'plt', necesario
↳ para poder usar
# mapas de color (colormaps) en la siguiente celda.
# Resultado esperado: no hay salida visible; matplotlib queda disponible como
↳ 'plt'.
```

```
[52]: # representación gráfica
corr_data.style.background_gradient(cmap=plt.get_cmap('RdYlGn'), axis=1)

# Línea 1: comentario descriptivo.
# Línea 2: .style.background_gradient aplica un gradiente de color de fondo a
↳ las celdas de la
# tabla según su valor numérico, usando el mapa de colores 'RdYlGn'
↳ (rojo-amarillo-
# verde, típicamente rojo para valores bajos/negativos y verde para
↳ valores altos/
```

```
#           positivos); axis=1 calcula el gradiente de color POR FILA
↳(comparando los valores
#           dentro de cada fila entre sí).
# Resultado esperado: la misma matriz de correlación, pero renderizada
↳visualmente con cada
#           celda coloreada según su magnitud (más roja cuanto más negativa/
↳baja, más verde
#           cuanto más positiva/alta dentro de su fila).
```

[52]: <pandas.io.formats.style.Styler at 0x2beadd37d50>

```
[53]: # correlación más negativa
mpg_data.drop(['año', 'origen'], axis=1).corr(numeric_only=True).idxmin()

# Línea 1: comentario descriptivo.
# Línea 2: calcula de nuevo la matriz de correlación (sin año/origen), y
↳idxmin() devuelve, para
#           CADA COLUMNA, la etiqueta de la FILA (es decir, el nombre de la otra
↳variable) con la
#           que tiene la correlación más negativa (el valor mínimo de esa
↳columna en la matriz).
# Resultado esperado: una Series donde el índice son los nombres de las
↳variables y el valor es
#           el nombre de la variable con la que cada una tiene la correlación
↳más negativa (por
#           ejemplo, 'mpg' probablemente señale a 'peso' o 'desplazamiento').
```

```
[53]: mpg           peso
cilindros          mpg
desplazamiento     mpg
potencia           mpg
peso               mpg
aceleracion        potencia
dtype: str
```

```
[54]: # correlación más positiva
mpg_data.drop(['año', 'origen'], axis=1).corr(numeric_only=True).idxmax()
↳#consigo misma....

# Línea 1: comentario descriptivo.
# Línea 2: idxmax() sobre la matriz de correlación devuelve, para cada columna,
↳la variable con
#           la que tiene la correlación MÁS ALTA; el comentario final ("consigo
↳misma....")
#           anticipa el problema: como la diagonal de la matriz siempre vale 1.0
↳(correlación de
```

```
#          una variable consigo misma), idxmax() siempre señalará a la propia
↳variable, lo cual
#          no es informativo.
# Resultado esperado: una Series donde cada variable aparece correlacionada
↳consigo misma (por
#          ejemplo, 'mpg' -> 'mpg'), un resultado trivial que no aporta
↳información útil.
```

```
[54]: mpg          mpg
cilindros          cilindros
desplazamiento     desplazamiento
potencia           potencia
peso               peso
aceleracion        aceleracion
dtype: str
```

```
[55]: # tabla similar con las correlaciones más positivas (evitar parejas del mismo
↳valor)
positive_corr = mpg_data.drop(['año', 'origen'], axis=1).corr(numeric_only=True)
np.fill_diagonal(positive_corr.values, 0)
display(positive_corr)
positive_corr.idxmax()

# Línea 1: comentario descriptivo indicando la solución al problema anterior.
# Línea 2: calcula la matriz de correlación de nuevo y la guarda en
↳'positive_corr'.
# Línea 3: np.fill_diagonal modifica IN PLACE el ndarray subyacente (.values)
↳de la matriz,
#          reemplazando todos los valores de la diagonal (que eran 1.0) por 0;
↳así se evita que
#          idxmax() señale trivialmente a la propia variable.
# Línea 4: muestra la matriz ya con la diagonal en 0.
# Línea 5: idxmax() ahora sí devuelve, para cada variable, la OTRA variable con
↳la que tiene la
#          correlación positiva más alta (ignorando la comparación consigo
↳misma).
# Resultado esperado: la matriz de correlación con ceros en la diagonal,
↳seguida de una Series
#          con parejas de variables genuinamente correlacionadas (por ejemplo,
↳'cilindros' y
#          'desplazamiento' probablemente se señalen mutuamente).
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[55], line 3
      1 # tabla similar con las correlaciones más positivas (evitar parejas del
↳mismo valor)
```



```

2 positive_corr = mpg_data.drop(['año', 'origen'], axis=1).
↳ corr(numeric_only=True)
----> 3 np.fill_diagonal(positive_corr.values, 0)
4 display(positive_corr)
5 positive_corr.idxmax()
6

File
↳ ~\anaconda3\envs\MachineLearning\Lib\site-packages\numpy\lib\_index_tricks_impl.
↳ py:923, in fill_diagonal(a, val, wrap)
920     step = 1 + (np.cumprod(a.shape[:-1])).sum()
922 # Write the value out into the diagonal.
--> 923 a.flat[:end:step] = val

ValueError: underlying array is read-only

```

```

[56]: positive_corr.style.background_gradient(cmap=plt.get_cmap('RdYlGn'), axis=1,
↳ vmin=-1.0, vmax=1.0)

# Línea 1: aplica el mismo gradiente de color visual que antes, pero esta vez
↳ fijando
#     explícitamente los límites de la escala de color con vmin=-1.0 y
↳ vmax=1.0 (el rango
#     teórico completo de un coeficiente de correlación), para que los
↳ colores sean
#     comparables de forma consistente independientemente de los valores
↳ máximos/mínimos
#     reales presentes en la tabla (y ahora con ceros en la diagonal en
↳ vez de unos).
# Resultado esperado: la matriz positive_corr renderizada con colores, usando
↳ una escala de
#     color fija de -1 (rojo) a 1 (verde), facilitando la comparación
↳ visual entre distintas
#     matrices de correlación.

```

[56]: <pandas.io.formats.style.Styler at 0x2beadfab4d0>

1.3.3 3.3 Ejercicios

- Ejercicios para practicar Pandas: <https://github.com/ajcr/100-pandas-puzzles/blob/master/100-pandas-puzzles.ipynb>